



**TECHNIQUES
DE L'INGÉNIEUR**

Réf. : **H2270 V1**

Langage C

Date de publication :
10 mars 1989

Cet article est issu de : **Archives**

par **Maurice FATHI**

Pour toute question :
Service Relation clientèle
Techniques de l'Ingénieur
Immeuble Pleyad 1
39, boulevard Ornano
93288 Saint-Denis Cedex

Par mail :
infos.clients@teching.com
Par téléphone :
00 33 (0)1 53 35 20 20

Document téléchargé le : **09/09/2024**

Pour le compte : **7200023220 - universite de lorraine // 193.50.135.4**

© Techniques de l'Ingénieur | Tous droits réservés

Langage C

par **Maurice FATHI**

Docteur de l'Université Pierre et Marie Curie

Licencié ès Sciences

Vice-Président de la société COSMIC

Animateur du groupe C à l'AFNOR

1. Introduction	H 2270 - 2
2. Normalisation et état actuel de C	— 2
3. Préprocesseur C	— 3
3,1 Définition et manipulation des macro-instructions	— 3
3,2 Inclusion de fichiers	— 4
3,3 Traitement conditionnel	— 4
3,4 Directives d'informations	— 4
4. Structure d'un programme C et exemples	— 5
4,1 Structure d'un fichier source C	— 5
4,2 Lien avec l'environnement	— 5
4,3 Jeu de caractères	— 5
4,4 Écriture des commentaires	— 6
4,5 Mots-clés du langage	— 6
4,6 Exemple de programme C	— 6
5. Objets manipulés en C	— 7
5,1 Classe d'un objet	— 7
5,2 Types de base du langage	— 7
5,3 Types tableaux et pointeurs	— 7
5,4 Types composés (énumérations, structures, unions)	— 8
5,5 Qualificateurs const et volatile	— 9
6. Opérateurs de C	— 9
6,1 Opérateurs unaires	— 9
6,2 Opérateurs binaires	— 10
6,3 Autres opérateurs	— 10
7. Fonctions	— 11
7,1 Définition et portée des fonctions	— 11
7,2 Prototypage	— 11
7,3 Appel de fonction	— 11
8. Instructions	— 12
9. Bibliothèques	— 13
10. Conclusion	— 13
Bibliographie	— 13

1. INTRODUCTION

1,1 Historique de C.

Le langage C est né aux Bell Laboratories vers 1972. Il a été conçu et développé par une seule personne : Denis Ritchie qui, à l'époque, travaillait en collaboration avec Ken Thompson sur le projet Unix (déjà). En fait, C est le descendant d'une suite de langages dont le premier pourrait être ALGOL 60, et le dernier le langage B; la figure 1 illustre cette généalogie.

Une des premières applications de C a été la réécriture quasi complète d'Unix, première pierre de fondation de l'indissociabilité de C et d'Unix. Unix est un système d'exploitation développé par les Bell Laboratories (cf. article spécialisé de ce traité).

Cette écriture d'Unix, à l'origine de sa portabilité, a également légué à C une étiquette de langage de développement de systèmes, ou de développement de logiciels systèmes ou de base. Si cet aspect de C a été suffisamment prouvé par Unix, il n'en est pas moins vrai que C a été utilisé (et le sera de plus en plus) dans de nombreux autres domaines, tels que les bases de données, les gestionnaires de communication, les outils bureautiques. Le langage C tend de plus en plus à devenir un langage universel.

De l'aveu même de son concepteur, D. Ritchie [1], C est un langage à vocation générale, et de « bas niveau », c'est-à-dire qu'il traite les objets connus de la plupart des calculateurs : les caractères, les nombres et les adresses. C n'offre aucune fonctionnalité telle que multiprogrammation, parallélisme, synchronisation, ou coroutines, et ce, malgré son étiquette de « langage système ».

C est un langage qui devait être simple afin que l'écriture d'un compilateur puisse être facilitée, et que le compilateur lui-même puisse être compact.

1,2 Raisons du succès initial de C.

Le succès initial de C est largement dû au système Unix. Il semblerait que Ritchie lui-même ait été étonné du succès de C sous Unix, et de sa prédominance sur des langages tels que ALGOL, FORTRAN ou encore PASCAL (dont la définition en 1970 par Wirth est contemporaine et même antérieure à celle de C). La diffusion gratuite d'Unix aux universités et aux établissements d'enseignement a été le vecteur de la diffusion de C : toute une génération de chercheurs et d'ingénieurs ont fait leurs premières armes avec Unix et donc avec C.

Néanmoins, ce facteur économique n'est certainement pas la seule explication du succès de C. Les facteurs techniques de la réussite de C sont, eux, à rechercher dans les propriétés de C, qui sont certainement celles que l'on a parfois qualifiées de *l'esprit C*.

Ce sont d'ailleurs ces qualités que les instances de normalisation ont mises en exergue de leurs travaux comme nous le verrons au paragraphe 2.

Il est bien difficile de résumer les caractéristiques d'un langage, ou de le définir en une seule phrase, néanmoins s'il fallait en choisir une pour C, la plus adaptée serait :

C est un assembleur portable de haut niveau.

C est un « assembleur » : en effet, C permet d'écrire très compact et proche de la machine; les concepts de base manipulés en C sont connus de tous les programmeurs en assembleurs.

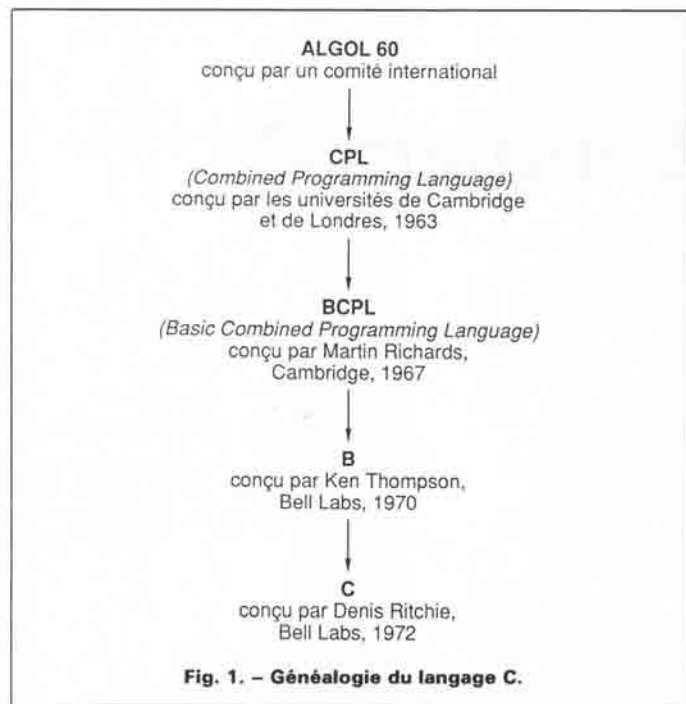


Fig. 1. – Généalogie du langage C.

C est « portable » ; cette assertion est amplement prouvée par le nombre de compilateurs disponibles, et leurs divers environnements de fonctionnement.

C est de « haut niveau » ; c'est en relation avec « assembleur » qu'il faut entendre ce qualificatif ; en effet C fournit des mécanismes de structuration et de contrôle qui sont plus puissants que ceux de la plupart des assembleurs. Comparé à d'autres langages structurés et de haut niveau, C est plutôt un langage de « bas niveau » tel que l'a dit Ritchie.

Une autre caractéristique de C (parfois décriée par ses détracteurs) est sa grande permissivité (comme un assembleur). En effet, la philosophie générale de C est que « le programmeur sait ce qu'il fait, et qu'en conséquence le langage ne devrait pas l'en empêcher ». Cette qualité extrêmement appréciée par les adeptes de C est certainement une des raisons de sa difficulté de pénétration dans certaines écoles de programmation.

Il est bon, lorsque l'on programme en C, de se fixer certaines règles de bon sens, afin de ne pas produire des programmes illisibles ou encore non maintenables [4].

2. NORMALISATION ET ÉTAT ACTUEL DE C

2,1 Processus de normalisation.

C est l'œuvre d'une personne unique : D. Ritchie. De ce fait, il n'a pas fait l'objet d'une définition ou d'une norme élaborée par un comité. La norme de fait a, pendant très longtemps, été l'ouvrage de B. Kernighan et D. Ritchie : [1]. Néanmoins, et malgré cette absence de norme, C a peu souffert des problèmes de dialectes divergents : la plupart des évolutions provenaient des Bell Laboratories et accompagnaient les évolutions d'Unix. Il est intéressant de noter que, dès le milieu des années soixante-dix, on a vu apparaître des compilateurs C pour de nombreux systèmes autres qu'Unix.

Néanmoins, c'est en juin 1983 que l'ANSI (American National Standards Institute) a démarré des travaux de normalisation de C dans le cadre du groupe X3J11. Le groupe de normalisation a, dès le départ, réuni de nombreux industriels impliqués dans l'écriture de compilateurs

C ou dans l'écriture de produits en langage C. Les travaux ont naturellement démarré avec pour base le *C Programming Language*, pour la partie langage, et le document établi par /usr/group (groupe d'utilisateurs d'Unix [2]) pour les bibliothèques. Les travaux de /usr/group étaient centrés sur la définition d'une norme de système d'exploitation compatible Unix.

Le groupe de normalisation a été au départ séparé en trois sous-comités :

- comité Langage, chargé de résoudre les problèmes de syntaxe et de sémantique.
- comité Bibliothèque, chargé, quant à lui, de la définition des bibliothèques, définition qui devait être faite en collaboration avec /usr/group (encore Unix!).
- comité Environnement, chargé tout particulièrement des aspects liés aux jeux de caractères, à la manipulation de fichiers, et à l'environnement système.

Les réunions du groupe de normalisation se sont tenues trimestriellement depuis le démarrage des travaux. La première proposition de norme (DRAFT) a vu le jour vers le milieu de l'année 1985. Les principes de base adoptés lors de la production de ce DRAFT étaient :

- ne pas favoriser une implantation existante particulière ;
- tenir compte de l'utilisation de C en tant que langage intermédiaire (générateurs de programmes, traducteurs de langages en C) ;
- cohabiter avec les outils existants et environnants (assembleurs, éditeurs de liens, etc.) ;
- permettre la génération de code efficace (rester près de la machine) ;
- permettre l'écriture de code dépendant système ;
- préserver la philosophie et l'esprit de C ;
- préserver dans la mesure du possible le code déjà écrit, c'est-à-dire assurer une compatibilité la plus grande possible avec les versions de langage existantes.

Les trois sous-comités sont à présent réunis en un seul groupe, qui est chargé de produire la version finale de la norme. Le DRAFT actuel a déjà passé deux phases de commentaire public, et devait passer la troisième phase de commentaire public fin 1988.

L'ANSI a déposé auprès de l'ISO (International Organization for Standardization) une demande de nouvelle question (*New Work Item*), qui constitue en fait le prélude d'une normalisation au niveau international. La volonté de l'ANSI d'aboutir à une norme acceptable par l'ISO est clairement affichée.

2,2 Quelques aspects de la norme.

Nous citerons ici, de façon non exhaustive, quelques-uns des acquis de la norme par rapport à la définition de Kernighan et Ritchie. Certaines de ces fonctionnalités paraîtront familières aux utilisateurs de C, et tout particulièrement aux utilisateurs Unix, qui en bénéficiaient déjà avec le compilateur C fourni avec Unix. Il est important de noter à ce titre qu'un des principes du groupe de normalisation de l'ANSI a été de produire une norme qui tienne compte de la pratique courante du langage ; le but de la norme n'était pas de réinventer C, mais de codifier ses caractéristiques les plus communément admises :

- ajout de nouveaux mots-clés : **enum**, **signed**, **void** (déjà connus sous Unix), **const** et **volatile**, totalement nouveaux et expliqués au paragraphe 5,5 ;

- prototypage des fonctions : C est un langage qui ne fournissait aucun mécanisme de vérification des paramètres de fonctions ; il est désormais possible en C ANSI de forcer une vérification grâce au «prototypage» (§ 7,2) ;

- possibilité de donner le même nom à des champs de structures et d'unions ;

- fonction de type **void**, c'est-à-dire ne retournant rien ; le compilateur se charge de vérifier qu'une telle fonction n'est pas utilisée comme une fonction standard ;

- définition du type **long double** : seuls le type **float** et **double** étaient précédemment définis (§ 5) ;

- passage des structures en paramètres par valeur ;

- fonctions retournant des structures ;

- différenciation très nette des pointeurs et entiers (**int**) : une pratique des plus courantes en C était de considérer qu'un entier et un pointeur avaient une structure identique et interchangeable ; en ANSI C, ces types sont maintenant décorrelés et la pratique précédente fortement déconseillée ;

- définition d'une bibliothèque d'entrée-sortie plus générale et indépendante d'Unix.

3. PRÉPROCESSEUR C

Tout compilateur C contient un utilitaire, appelé préprocesseur, qui agit au niveau des fichiers contenant des programmes C et fournit les fonctionnalités suivantes :

- définition et manipulation de macro-instructions avec ou sans arguments ;
- inclusion de fichiers ;
- traitement conditionnel ;
- directives, qui permettent d'adapter le compilateur à un environnement particulier (cf. § 3,4).

Le préprocesseur C évalue les fichiers source ligne par ligne, et reconnaît les lignes commençant par le caractère «#» comme des lignes contenant les directives de prétraitement qui lui sont propres. Les directives sont définies à l'avance, il n'est pas possible «d'enrichir» le préprocesseur avec de nouvelles directives propres à chaque utilisateur. Le préprocesseur ne s'intéresse qu'aux lignes contenant les directives, les autres sont soit ignorées, soit traitées en fonction des directives précédemment rencontrées. Les directives du préprocesseur ne sont pas des mots-clés du langage, il est donc possible de les employer séparées de leur caractère de reconnaissance particulier (#).

Nous allons passer rapidement en revue les différentes directives possibles.

3,1 Définition et manipulation des macro-instructions.

La directive :

define <ident> <définition>

définit <ident> comme une macro-instruction. Dans toutes les lignes sources qui suivent, le préprocesseur remplacera toute occurrence de <ident> par le contenu de <définition>.

Exemple :

```
# define PI 3.141592
```

Toutes les occurrences de PI dans la suite du programme seront remplacées par la constante 3.141592. Écrire :

```
x = PI
```

est donc strictement équivalent à écrire :

```
x = 3.141592.
```

On peut également définir des macro-instructions avec arguments. Une macro-instruction avec argument est de la forme :

```
# define <ident> (<args>) <définition>
```

<ident> est alors une macro-instruction avec paramètres (<args>). Toute occurrence de <ident> suivie d'une liste d'arguments entre parenthèses sera remplacée par le contenu de <définition> après remplacement des paramètres.

Exemple :

```
# define sum(x, y) (x + y)
```

```
# define max(a, b) (a < b) ? b : a;
```

Écrire `sum(1, 2)` devient alors équivalent à écrire `(1 + 2)`.

Écrire `max(a, b)` permet alors d'obtenir le maximum de a et de b.

Il est possible d'imbriquer les appels de macro-instructions et ce même de façon récursive :

```
sum(sum(3, 2), 4) est équivalent à : ((3 + 2) + 4)
```

Le nombre d'arguments présents dans la définition d'une macro-instruction doit être respecté lors de l'appel de cette même macro-instruction.

Il est également possible de mettre fin à la définition d'une macro-instruction grâce à la directive **# undef <ident>**.

Exemple :

```
# define toto titi
```

```
x = toto;
```

```
# undef toto
```

```
y = toto;
```

une fois traitée par le préprocesseur, la séquence précédente est équivalente à :

```
x = titi;
```

```
y = toto;
```

3,2 Inclusion de fichiers.

Les directives suivantes :

```
# include <nom_de_fichier>
```

```
# include « nom_de_fichier »
```

ont pour effet leur remplacement par le contenu du fichier désigné par `nom_de_fichier`. Les deux syntaxes permettent de distinguer les fichiers propres à l'utilisateur de ceux qui sont fournis en standard avec tous les compilateurs. Il faut utiliser :

```
# include <nom_de_fichier>
```

pour inclure un fichier standard fourni avec le compilateur, et

include « nom_de_fichier » pour inclure un fichier propre à un utilisateur ou à une application.

3,3 Traitement conditionnel.

Diverses directives permettent de rendre conditionnel le fonctionnement du préprocesseur, c'est-à-dire d'éliminer totalement une partie du fichier source traité.

La directive :

```
# ifdef <ident>
```

a pour effet, si <ident> n'est pas défini comme une macro-instruction, d'éliminer toutes les lignes sources jusqu'à la rencontre d'une directive inverse qui doit être l'une des suivantes : **# endif**, **# else** ou **# elif**.

La directive :

```
# ifndef <ident>
```

a un effet identique à la précédente, les lignes sources n'étant éliminées que si la macro-instruction <ident> est définie.

La directive :

```
# else
```

permet d'inverser l'effet d'un **# ifdef** ou d'un **# ifndef** précédent.

La directive :

```
# endif
```

permet de mettre fin au **# ifdef**, **# ifndef** ou le moins antérieur.

La directive :

```
# elif
```

est une forme abrégée de la séquence suivante :

```
# else
```

```
# if
```

Il existe également une forme plus élaborée de traitement conditionnel, la directive :

```
# if <expr>
```

permet de conditionner un traitement à la valeur d'une expression entière désignée par <expr>, dont l'écriture est identique à celle des expressions entières manipulées par C. **# if** fonctionne comme **# ifdef** excepté qu'elle teste si <expr> est nul au lieu de tester si <ident> est une macro-instruction.

Exemples :

```
# ifdef PI
```

```
x = PI / 2.;
```

```
# else
```

```
x = 3.14 / 2.;
```

```
# endif
```

si la macro-instruction PI n'est pas définie, on utilisera l'approximation 3.14.

3,4 Directives d'informations.

Les directives dites d'informations sont :

```
# line <constante>
```

qui permet de modifier le numéro de ligne qui sera utilisé par le compilateur par la suite ;

```
# pragma
```

qui permet de communiquer au compilateur des informations dont l'interprétation est laissée à celui-ci. Les pragma constituent en fait un

mécanisme d'échappement permettant d'implanter des traitements spécifiques et non portables.

Le préprocesseur est un élément très important des compilateurs C. Il permet, tout en ne surchargeant pas le langage lui-même, d'offrir les mécanismes nécessaires à un paramétrage de l'environnement de compilation et de développement de programmes.

4. STRUCTURE D'UN PROGRAMME C ET EXEMPLES

Nous allons dans ce paragraphe examiner la structure d'un programme C, c'est-à-dire la façon dont se présente un programme C, le jeu de caractères utilisé par le langage, les règles d'écriture des commentaires, ainsi que ses mots-clés. C est un langage compilé qui admet, contrairement à PASCAL (du moins dans sa version ISO), la compilation séparée.

Un programme C se présente donc tout naturellement comme une suite de fichiers sources contenant les divers constituants de l'application finale, c'est-à-dire les données et les actions ou algorithmes.

La compilation d'un fichier source C fournit un module objet, et c'est l'édition de liens de tous ces objets avec l'environnement d'exécution de C qui constituera le programme final. La figure 2 schématise cette structure.

4.1 Structure d'un fichier source C.

La structure générale d'un fichier C est la suivante : on trouve d'abord la déclaration ou la définition des variables qui seront utilisées, puis une suite d'actions ou fonctions qui feront partie du corps de programme.

Les variables C sont partagées en deux classes, les **variables globales** et les **variables locales**. Les variables globales sont celles qui seront visibles dans toute l'application et donc dans tous les fichiers la constituant ; les variables locales sont celles qui seront seulement visibles dans le fichier courant. En ce qui concerne les variables globales, il est important de distinguer la définition, qui est en fait l'énoncé qui va donner corps à la variable, de la déclaration qui, elle, se borne à préciser les règles de manipulation applicables à cette même variable.

Toute variable C doit être déclarée avant son utilisation ; le compilateur ne gère pas les conflits éventuels de déclaration entre les divers fichiers. Les règles applicables à une variable sont celles propres à la déclaration visible dans le fichier courant. Pour plus de précisions, le lecteur se reportera au paragraphe 5.

De même que les variables, les fonctions C sont partagées en deux classes ; les fonctions globales sont celles qui seront accessibles dans tous les fichiers constituant le programme ; les fonctions locales sont celles qui ne sont accessibles que dans le fichier où elles ont été définies. Par défaut, une fonction C est globale. Nous reviendrons sur ces divers points au paragraphe 7.

La notion de procédure n'existe pas en C ; seule la notion de fonction existe.

Il existe en C une fonction privilégiée, c'est la fonction principale qui représente le point d'entrée du programme. Par convention cette fonction s'appelle **main** et doit être présente dans l'un des fichiers consti-

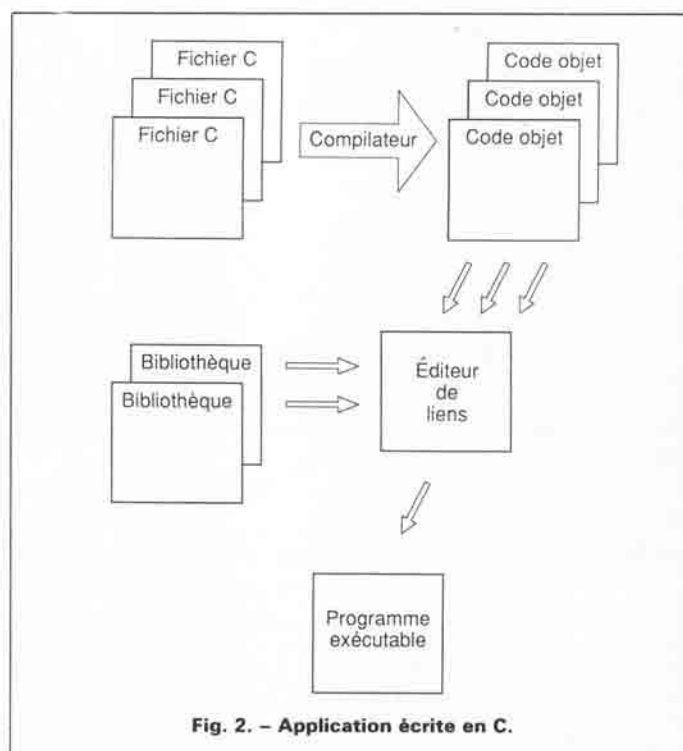


Fig. 2. - Application écrite en C.

tuant l'application. Cette fonction **main** permet d'assurer un lien entre un programme C et son environnement d'exécution. Ce lien est explicité dans le paragraphe suivant.

4.2 Lien avec l'environnement.

Tout programme C démarre son exécution à la fonction **main**. L'environnement d'exécution C garantit que lors de l'appel de **main** les conditions suivantes seront réalisées :

- les paramètres d'activation du programme sont disponibles comme paramètres de **main** ; en fait **main** reçoit en argument une suite de chaînes de caractères qui sont dérivées de la ligne de commande d'activation du programme ;
- les entrées-sorties sont possibles sur le périphérique à partir duquel a eu lieu l'activation du programme C.

L'on remarquera qu'il n'est donc pas nécessaire en C d'effectuer des traitements spéciaux afin d'effectuer une impression ou d'exécuter une entrée de donnée.

De même que l'environnement d'exécution communique à la fonction **main** des arguments, celle-ci peut renvoyer un compte rendu qui sera ou non exploité par l'environnement.

La plupart de ces concepts sont en fait des héritages d'Unix qui, comme on l'a vu, a été le berceau du langage C.

4.3 Jeu de caractères.

Le jeu de caractères utilisé en C est un sous-ensemble du jeu ASCII (American Standard Code For Information Interchange), à savoir :

- les caractères majuscules **ABCDEFGHIJKLMNPOQRSTUVWXYZ** et les caractères minuscules **abcdefghijklmnopqrstuvwxyz** utilisés pour former les identificateurs et les mots-clés ;
- les chiffres décimaux **0123456789** utilisés pour former les constantes entières et flottantes ;
- le caractère souligné **_**, qui peut être utilisé dans les identificateurs ;

- le caractère barre oblique inverse \, qui est utilisé comme caractère d'échappement ;
- les caractères de ponctuation suivants, utilisés pour former des séparateurs et des opérateurs : `[]{}()<>!#%^&*+--=:;"/.|-?;` ;
- les caractères suivants utilisés comme séparateurs : **tabulation horizontale, retour chariot, tabulation verticale, saut de page et nouvelle ligne.**

4,4 Écriture des commentaires.

Les commentaires en C sont libres, et peuvent apparaître n'importe où dans un programme ; ils sont délimités par les séquences respectives suivantes : `/*` pour début de commentaire et `*/` pour les fins de commentaires. Les formes suivantes sont donc parfaitement légales :

```
/* ceci est un commentaire */
ou encore
/* ceci est un
commentaire
*/
ou encore
i = /* affecter j dans i */ j ;
Il n'y a pas d'imbrication des commentaires en C.
```

4,5 Mots-clés du langage.

C est un langage qui utilise des mots-clés. Ceux-ci sont en nombre assez restreint pour respecter le vœu des concepteurs, qui souhaitaient que le langage soit simple. La liste des mots-clés est donnée ci-après, telle qu'elle ressort des travaux de normalisation du groupe ANSI :

auto	extern	sizeof
break	float	static
case	for	struct
char	goto	switch
const	if	typedef
continue	int	union
default	long	unsigned
do	register	void
double	return	volatile
else	short	while
enum	signed	

4,6 Exemple de programme C.

Le programme ci-après est un programme de tri récursif.

```
#include <std.h>
/*
 *
 * Programme de tri
 *
 */
```

```
/* nombres d'éléments du tableau
 */
#define NBMOT 22
/* tableau à trier */
extern short suite [NBMOT] =
    { '5','1','7','3','9','2','4','6','0','0','A','C','D','E','F',
      '1','B','0','K','W','R','Y' };
/*
 * Programme principal
 */
main ()
/* appel de la fonction de tri
 * en C, les éléments de tableaux
 * s'adressent à partir de l'indice 0
 */
    trier(0, NBMOT-1);
/*
 * fonction de tri
 */
trier(gauche, droite)
    int gauche, droite;
    { int i, j;
      short x, w;
      i = gauche;
      j = droite;
/* >> est l'opérateur de décalage à droite
 * >> 1 signifie "décaler à droite de 1 bit", ce qui
 * revient à diviser par deux mais produit un programme plus
 * optimisé */
      x = suite[(i + j) >> 1];
      do {
          while (suite[i] < x)
              ++i; /* ++i signifie incrémenter la variable i
                  * de 1 avant de s'en servir */
          while (x < suite[j])
              --j; /* --j signifie décrémenter la variable
                  * j de 1 avant de s'en servir */
          if (i <= j) /* <= signifie inférieur ou égal */
              { w = suite[i];
                suite[i] = suite[j];
                suite[j] = w;
                ++i;
                --j;
              } while (i <= j);
          if (gauche < j)
              trier(gauche, j);
          if (droite > i)
              trier(i, droite);
      } while (gauche < droite);
    }
```

5. OBJETS MANIPULÉS EN C

C se voulant un langage simple, les objets qu'il manipule sont eux-mêmes relativement simples. Il est néanmoins possible de construire des objets complexes ou extrêmement sophistiqués avec les mécanismes de structuration de données du langage [6].

Nous allons nous attacher à décrire les objets manipulés par C et les mécanismes de structuration offerts par le langage.

Un objet C est caractérisé par les informations suivantes :

- sa classe, c'est-à-dire en fait sa durée de vie et sa visibilité ;
- son type et un qualificateur optionnel qui définissent les opérations qui pourront lui être appliquées ;
- son nom, c'est-à-dire l'identificateur qui lui est attaché ;
- sa valeur de définition, ou valeur initiale.

Ces diverses informations définissent complètement un objet manipulable par le langage, ainsi que l'usage légal qui peut en être fait.

Il n'est pas nécessaire de spécifier toutes ces informations pour définir un objet, des valeurs par défaut sont disponibles pour compléter les informations fournies par le programme.

5,1 Classe d'un objet.

La classe d'un objet est spécifiée à l'aide des mots-clés suivants.

- **extern** qualifie une variable globale, accessible par tous les fichiers de l'application, et dont la durée de vie est égale à celle du programme.
- **static** qualifie une variable dont la durée de vie est égale à celle du programme, mais dont la visibilité est limitée soit à une fonction (variables rémanentes), soit à un des fichiers de l'application.
- **auto** caractérise les variables locales à un bloc, leur durée de vie est égale à celle du bloc ; elles sont allouées et détruites automatiquement (d'où la classe « auto »). C'est, par exemple, la classe par défaut des variables locales à des fonctions.
- **register** classifie une variable comme devant être placée dans un des registres de la machine supportant l'exécution du programme. Cette notion est relativement originale à C. Le registre utilisé est laissé à l'initiative du compilateur. L'intérêt des variables **register** est l'optimisation qui résulte de l'utilisation de registres par rapport à l'utilisation de mémoire.
- **typedef** permet de construire un nouveau type. Le type ainsi défini peut alors être utilisé comme un type de base. Sachant que le type entier se déclare en C avec le mot-clé **int** et qu'il est possible de spécifier avec le mot-clé **unsigned** qu'une variable ne peut prendre que des valeurs positives, la ligne suivante :
typedef unsigned int entier_non_signé
permet de définir le nouveau type **entier_non_signé**.

Par défaut, toute variable déclarée dans un bloc sera de classe **auto**, et toute variable déclarée en dehors de tout bloc aura pour classe **extern**.

La classe d'un objet ne définit pas les opérations ou les opérateurs qui pourront lui être appliqués. Cette information est donnée par le type de l'objet.

5,2 Types de base du langage.

Les types de base en C sont simples. Il est amusant de noter que C ne fournit par exemple pas de type booléen (ou logique), pas plus qu'il ne fournit de type caractère ou chaîne de caractères. En fait, les types de base en C sont entier ou flottant, et il est possible de leur attribuer une taille ou un signe tous deux optionnels.

La spécification de la taille se fait à l'aide des mots-clés : **short** et **long** ; les mots-clés **signed** et **unsigned** permettent d'indiquer si une variable doit être traitée comme signée ou non. Les entiers sont déclarés à l'aide des mots-clés **char** et **int** ; les flottants à l'aide des mots-clés **float** et **double**.

Un **char** est un entier permettant de coder le jeu de caractères utilisé par le langage, typiquement un octet.

Les combinaisons permises et leurs significations sont énumérées ci-dessous :

- **signed char** entier signé codé sur un octet ;
- **unsigned char** entier non signé codé sur un octet ; **char** tout seul est équivalent soit à **signed char**, soit à **unsigned char** ; le choix est laissé à l'initiative du compilateur ;
- **short**, **short int**, **signed short int**, et **signed short** sont identiques et indiquent un entier signé dont l'encombrement minimal est de 16 bits ;
- **unsigned short** et **unsigned short int** sont identiques et spécifient un entier non signé dont l'encombrement minimal est de 16 bits ;
- **int** et **signed int** sont équivalents, et spécifient un entier dont la taille est celle « naturelle » à la machine supportant l'exécution du programme ;
- **unsigned int** et **unsigned** définissent tous deux un entier non signé ;
- **long**, **long int**, **signed long** et **signed long int** sont identiques et définissent un entier signé dont l'encombrement minimal est de 32 bits ;
- **unsigned long** et **unsigned long int** spécifient un entier non signé dont l'encombrement minimal est de 32 bits ;
- **float** spécifie le type flottant d'encombrement minimal ;
- **double** spécifie un type flottant dont la précision est supérieure à celle de **float** ;
- **long double** spécifie le type flottant de précision maximale.

5,3 Types tableaux et pointeurs.

En plus des types de base cités précédemment, C fournit des types plus élaborés, tels que tableaux et pointeurs. La notion de pointeur est une notion essentielle en C. Elle apparaît certes dans d'autres langages, mais est très intensivement utilisée en C. C'est aussi la notion la plus difficile à faire passer, lorsque l'on présente C à des utilisateurs de langages de haut niveau.

5,31 Tableaux. – Un tableau est une suite ordonnée et consécutive d'éléments de même type. Les tableaux peuvent avoir une ou plusieurs dimensions. La déclaration d'un tableau se fait de la façon suivante :

<type> <nom> [<dim1>] ... [<dim2>]

Par exemple la déclaration d'un tableau de 10 entiers de nom **tab** s'écrit :

```
int tab[10];
```

La déclaration d'un tableau à trois dimensions s'écrit :

```
int tab[10][20][30];
```

Les indices sont indiqués entre crochets, et chaque indice est inscrit dans une paire de crochets qui lui est propre.

En C, les éléments d'un tableau s'adressent à partir de l'indice zéro, et non pas de l'indice 1. Dans l'exemple suivant :

```
int a[3];
```

les éléments du tableau **a** s'écrivent :

```
a[0], a[1] et a[2].
```

Il est naturellement possible de déclarer des tableaux dont le type des éléments est plus élaboré qu'un type simple. Les règles de déclaration et d'accès restent les mêmes.

Enfin, il faut savoir que C ne vérifie en aucune façon les débordements de tableaux. Il est donc parfaitement légal de déclarer par exemple un tableau à trois éléments et de demander à accéder au vingtième.

5,32 Pointeurs. — Un pointeur est un objet qui peut contenir l'adresse d'un autre objet. En fait, il faut définir un pointeur comme l'adresse d'un objet et le type de l'objet qui est ainsi pointé. Les règles de manipulation des pointeurs sont relativement strictes (surtout dans le cadre de la proposition de norme de l'ANSI).

La déclaration d'un pointeur se fait de la façon suivante :

```
<type> * <nom>;
```

Le caractère ***** est le caractère qui permet de reconnaître la déclaration d'un pointeur. Par exemple :

```
int *pnt;
```

déclare un pointeur vers un objet qui sera du type entier. La valeur du pointeur est dénotée par **pnt**, alors que la valeur de l'objet est dénotée par ***pnt**.

Ainsi dans l'exemple suivant :

```
int i, j;
int *pnt;
pnt = &i;
*pnt = j;
```

l'instruction **pnt = &i** affecte au pointeur **pnt** l'adresse de la variable **i**, alors que l'instruction ***pnt = j** affecte à la variable pointée par **pnt**, c'est-à-dire à **i**, la valeur **j**.

Enfin, il est presque partout possible de remplacer un pointeur par le nom d'un tableau, ce qui a souvent concouru à la confusion des utilisateurs non initiés du langage.

5,4 Types composés.

C fournit des mécanismes de structuration de données, qui permettent de construire des types d'objets plus élaborés que ceux que nous avons vus jusqu'à présent.

Les types composés disponibles en C sont : les **enum** (énumérations), les **struct** (structures), les **union** (unions). Nous allons rapidement passer en revue ces divers types composés.

5,41 Énumérations. — Une énumération permet de définir une variable, dont les valeurs possibles peuvent être définies de façon énumé-

rative. Les valeurs possibles citées dans la définition de l'énumération deviennent alors des constantes utilisables avec l'énumération ainsi définie.

En fait les énumérations sont codées en interne par le compilateur, à l'aide d'entiers, mais ce mécanisme est totalement transparent à l'utilisateur.

Les énumérations se déclarent à l'aide du mot-clé **enum**.

Exemple :

```
enum | ROUGE, JAUNE, VERT | feu;
```

déclare l'énumération **feu** dont les valeurs possibles sont **ROUGE**, **JAUNE** ou **VERT**.

5,42 Structures. — Une structure est le regroupement de plusieurs objets en un seul, qui pourra soit être manipulé comme un tout, soit dont on pourra à loisir manipuler l'un des composants ou champs. C'est une notion équivalente à celle de structure en PL/I, ou d'enregistrement en COBOL ou encore d'article en PASCAL.

Les champs d'une structure peuvent être quelconques. Il est ainsi possible de définir des structures de structures ou des structures de tableaux et ce, à un niveau d'imbrication quelconque.

La structure permet également la définition d'un nouveau type C, celui de **bitfield** ou de champ de bits. Il est possible dans une structure, et seulement dans une structure, de définir des champs qui sont une suite de bits consécutifs, dont le nombre est défini lors de la définition du champ correspondant. Les divers champs de bits consécutifs sont dans la mesure du possible alloués dans le même mot machine.

Les structures sont déclarées via le mot-clé **struct**. Voici quelques exemples de déclaration de structure :

```
struct |
char nom[50];
char rue[50];
char ville[30];
long code_postal;
int credit;
champ_de_bit1 : 4; /* champ de 4 bits */
champ_de_bit2 : 12; /* champ de 12 bits */
int *reste; /* pointeur */
} client1;
```

crée l'objet **client1** qui est composé des divers champs indiqués ci-dessus ; l'accès à l'un des champs s'écrit à l'aide du caractère **point**. Ainsi le code postal de **client1** peut être obtenu en écrivant :

```
client1.code_postal
```

Il est possible avec le mot-clé **struct** de définir un nouveau type. Ainsi, si l'on voulait définir un type **client**, et non pas décrire un objet de type **client**, l'on écrirait :

```
struct client |
char nom[50]; /* tableau */
char rue[50];
char ville[30];
long code_postal; /* entier long */
int credit;
champ_de_bit1 : 4; /* champ de 4 bits */
champ_de_bit2 : 12; /* champ de 12 bits */
int *reste; /* pointeur */;
```

La déclaration d'un objet de type client s'écrit alors :

```
struct client client1;
```

La déclaration d'un pointeur vers un objet de type client s'écrit :

```
struct client *pntcli;
```

Lorsque l'on utilise un pointeur, l'accès aux divers champs de la structure pointée s'obtient avec l'opérateur `->`. Ainsi si l'on veut connaître la valeur du champ `code_postal` de la structure pointée par `pntcli`, l'on écrit :

```
pntcli->code_postal
```

La notion de structure est très souvent utilisée en C. Elle est certainement au centre du langage avec celle de pointeur.

La notion de structure, si elle permet bien la définition d'un objet composé, ne le permet que dans le cas où la description de cet objet est fixe ; elle ne permet pas de décrire des objets dont la description peut varier dans certaines limites en fonction de certaines conditions. C'est la notion d'**union** qui permet ce genre de mécanisme.

5,43 Unions. — C'est la description d'un objet, sous la forme de ses différentes alternatives. Si la structure permet d'interpréter un objet en fonction d'une description figée, l'union permet, quant à elle, de l'interpréter selon une description variable.

Une union permet d'interpréter une même zone mémoire selon des descriptifs différents.

La déclaration d'une union se fait avec le mot-clé **union** suivi d'une description analogue à celle d'une structure. Les divers champs sont alors interprétés comme les différentes descriptions possibles de l'objet recouvert par cette union.

L'exemple ci-dessous permet d'illustrer cette notion :

```
union |
int ival;
float fval; | mem;
```

déclare l'union `mem` qui peut contenir soit un entier, soit un nombre flottant ; `mem.ival` permet d'interpréter la zone mémoire recouverte par `mem` comme un entier, alors que `mem.fval` permet de l'interpréter comme un nombre flottant.

La déclaration d'une **union** de même que l'accès à l'un de ses champs se font de manière analogue à celle d'une structure et il est naturellement possible de mélanger les deux comme dans l'exemple suivant :

```
struct objet |
int ival;
char nom[50];
short *suite;
union |
char carac;
int valeur;
float flottant;
| u1; |;
```

5,5 Qualificateurs **const** et **volatile**.

Les travaux de normalisation de l'ANSI ont introduit deux nouveaux qualificateurs **const** et **volatile**, qui sont utilisables sur tous les objets. Le qualificateur **const** permet de spécifier que l'objet auquel il s'appli-

que est un objet constant, c'est-à-dire qu'il ne peut pas être modifié par l'exécution du programme, le compilateur pouvant alors décider de tirer parti de ce fait.

Le qualificateur **volatile** permet de spécifier la situation inverse. Il s'applique à un objet dont la valeur est volatile (tel qu'un registre d'entrée-sortie), c'est-à-dire qui peut être modifiée à tout moment par un événement externe au programme. Le compilateur doit alors éviter toute optimisation relative à la manipulation de cette variable.

6. OPÉRATEURS DE C

C offre un riche éventail d'opérateurs pour spécifier des actions sur des entiers, des flottants, des pointeurs et occasionnellement des types composés. Les opérateurs se classent en opérateurs d'adresse, opérateurs unaires et opérateurs binaires. Les opérateurs d'adresse sont les plus prioritaires, évalués de gauche à droite par rapport au terme affecté ; puis les opérateurs unaires sont évalués de droite à gauche, en commençant par les postfixes `++` et `--`. Les opérateurs binaires sont ensuite évalués selon leurs priorités respectives.

On peut utiliser les parenthèses pour imposer un ordre d'évaluation, sans avoir peur de modifier la signification de l'expression par une redondance de parenthèses ; `f(p)` est identique à `f((p))` et à `((f(p)))`.

Certains opérandes sont classés « G-valeur », c'est-à-dire qu'on peut les positionner à la gauche de l'opérateur d'affectation. La plus simple G-valeur est un identificateur, mais toute expression dont l'évaluation permet de localiser un objet déclaré est une G-valeur. Les expressions scalaires sont classées « D-valeur », c'est-à-dire qu'on peut les positionner à droite de l'opérateur d'affectation. Toutes les G-valeurs sont aussi des D-valeurs.

Pratiquement tous les opérateurs C ne manipulent que des types scalaires, c'est-à-dire les types de base, les pointeurs ou les champs de bits ; les structures sont autorisées dans le projet de norme ANSI, mais seulement pour l'affectation.

6,1 Opérateurs unaires.

***p** `p` doit être du type pointeur vers ... La valeur obtenue est celle de l'objet pointé par `p`, et de type ...

&x `x` doit être une G-valeur. Le résultat est un pointeur qui pointe vers `x`. Le type résultant est pointeur vers un objet du type de `x`.

+x `x` doit être de type entier ou réel. Le résultat est une D-valeur de même valeur et type que `x`.

-x `x` doit être de type entier ou réel. Le résultat est une D-valeur du même type que `x` et de la valeur opposée.

++x `x` doit être une G-valeur scalaire ; `x` est incrémenté de 1. Le résultat possède la nouvelle valeur et le type de `x`.

--x `x` doit être une G-valeur scalaire ; `x` est décrémenté de 1. Le résultat possède la nouvelle valeur et le type de `x`.

x++ `x` doit être une G-valeur scalaire ; `x` est incrémenté de 1. Le résultat possède l'ancienne valeur et le type de `x`.

x-- `x` doit être une G-valeur scalaire ; `x` est décrémenté de 1. Le résultat possède l'ancienne valeur et le type de `x`.

~x `x` doit être un entier. Le résultat est le complément à un de `x`, du même type que `x`.

!x x doit être scalaire. Le résultat est l'entier 1 si x est nul, et l'entier 0 sinon.

(<a-type>)x <a-type> est une déclaration de type scalaire sans le nom associé, par exemple (char *). Le résultat est une D-valeur obtenue en convertissant x dans le type <a-type>. Cet opérateur est appelé « opérateur de conversion » (voir Déclarations).

sizeof x, sizeof (<a-type>). Le résultat est une D-valeur entière égale à la taille en octets de x, ou la taille en octets d'un objet du type <a-type>.

6,2 Opérateurs binaires.

En général, le type d'un opérateur binaire est le plus grand des types de ses opérandes, l'opérande de type le plus petit étant converti dans le type le plus grand. Il existe un ordre implicite de grandeur parmi les types arithmétiques, à savoir : char, unsigned char, short, bitfield (si int est équivalent à short), unsigned short, long, bitfield (si int est équivalent à long), unsigned long, float et double ; double est le type le plus grand.

Les opérateurs sont examinés dans l'ordre des priorités décroissantes.

x*y Les deux opérandes doivent être arithmétiques (entier ou réel). Le résultat est le produit de x par y dans le type de l'opérande de plus grand type.

x/y Les deux opérandes doivent être arithmétiques (entier ou réel). Le résultat est le quotient de x par y dans le type de l'opérande de plus grand type. Cet opérateur a la même priorité que *.

x%y Les deux opérandes doivent être entiers. Le résultat est le reste de x divisé par y dans le type de l'opérande de plus grand type. Cet opérateur a la même priorité que *.

x+y Si l'un des opérandes est du type pointeur vers ..., l'autre opérande doit être du type entier, et sa valeur sera multipliée par la taille en octets du type ... avant d'être ajoutée au pointeur pour obtenir un résultat du type pointeur. Sinon, les deux opérandes doivent être arithmétiques et, dans ce cas, le résultat est égal à la somme des deux opérandes, dans le type le plus grand.

x-y Si x est du type pointeur vers ... et y d'un type entier, y est d'abord multiplié par la taille du type ... puis soustrait au pointeur x pour donner un résultat du type pointeur. Si les deux opérandes sont du type pointeur vers ..., et que les deux types pointés sont de même taille, le résultat est du type entier et égal à la différence des deux opérandes divisée par la taille en octet de ... Sinon, les deux opérandes doivent être arithmétiques. Le résultat est égal à la différence des deux opérandes, dans le type le plus grand. L'opérateur a même priorité que +.

x<<y Les deux opérandes doivent être entiers. Le résultat est égal à x décalé à gauche de y bits, et est du type de x.

x>>y Les deux opérandes doivent être entiers. Le résultat est égal à x décalé à droite de y bits, et est du type de x. Si le type résultant est non signé, le décalage s'opère sans duplication du bit de signe. Si le type résultant est signé, le décalage conserve le signe.

x<y, x<=y, x>y, x>=y Si l'un des deux opérandes est du type pointeur vers ... et que l'autre est du type entier, l'entier est multiplié comme pour une addition avant d'effectuer la comparaison des deux opérandes ainsi obtenus. Sinon, si les deux opérandes sont du type pointeur vers ..., ils sont comparés comme des entiers non signés. Sinon x et y doivent être arithmétiques, et les types

sont ajustés au plus grand avant comparaison. Le résultat est l'entier 1 si la relation est satisfaite, et l'entier 0 sinon.

x==y, x!=y Les opérandes sont traités comme pour les relations précédentes, puis comparés en égalité (==) ou en inégalité (!=). Le résultat est l'entier 1 si la relation est satisfaite, et l'entier 0 sinon.

x&y Les deux opérandes doivent être entiers (C n'a pas de type logique). Le résultat est égal au et logique de x et de y.

x^y Les deux opérandes doivent être entiers. Le résultat est égal au ou exclusif de x et de y.

x|y Les deux opérandes doivent être entiers. Le résultat est égal au ou logique de x et de y.

x&&y Les deux opérandes doivent être scalaires (C n'a pas de type booléen). Si x est nul, le résultat est l'entier 0, et y n'est pas évalué. Sinon, le résultat est l'entier 1 seulement si x et y sont nuls.

x||y Les deux opérandes doivent être scalaires. Si x est non nul, le résultat est l'entier 1, et y n'est pas évalué. Sinon le résultat est l'entier 0 si x et y sont nuls.

t?x:y Si t, qui doit être scalaire, est non nul, le résultat est x. Sinon, le résultat est y. Un seul des deux opérandes est évalué.

x=y Si les deux opérandes sont scalaires, x doit être une G-valeur ; y est converti dans le type de x et est affecté à x. Si x est un pointeur vers ..., y peut être un pointeur vers ... ou un entier (qui n'est pas multiplié par la taille dans ce cas). Sinon les deux opérandes doivent être arithmétiques. Le résultat est une D-valeur égale à la valeur affectée et du type de x.

x*=y, x/=y, x%=y,

x+=y, x-=y, x<<=y,

x>>=y, x&=y, x^=y, x|=y Chaque opération « x op = y » est équivalente à « x = x op y », sauf que x n'est évalué qu'une seule fois, et que « x op y » doit être du même type que x, c'est-à-dire que « x -= y » n'est pas permis si x et y sont des pointeurs. La forme « =op » est aussi permise pour des raisons historiques, mais il ne faut pas mettre d'espaces entre le « = » et l'op. Ces opérateurs ont la même priorité que =.

x,y Les deux opérandes doivent être scalaires ; x est évalué en premier, puis y. Le résultat est égal à la valeur et au type de y. Attention au fait que les virgules dans les listes d'arguments sont des séparateurs et non pas des opérateurs. Ainsi f(a,b,c) représente trois arguments, alors que f(a,(b,c)) n'en représente que deux, le second étant c, évalué après b.

6,3 Autres opérateurs.

a[i] Cette opération est entièrement équivalente à « *(a + i) ». Si a est du type tableau de ..., et i du type entier, l'effet le plus direct est d'obtenir le ième élément de a, qui est du type ...

x.m x doit être une G-valeur du type struct ou union et contenant un élément de nom m (m ne peut jamais être une expression). La valeur obtenue est celle de l'élément m de la structure x, dont le type est spécifié par m.

p->m p doit pouvoir être converti en un pointeur vers une structure ou une union contenant un membre de nom m (m ne peut jamais être une expression). La valeur obtenue est celle du membre m de la structure pointée par p, dont le type est spécifié par m.

7. FONCTIONS

Une fonction en C est l'équivalent d'une fonction PASCAL ou encore d'une procédure ALGOL ou PL/I. Tout programme C est une suite de fonctions, dont l'une d'elles (la fonction `main()`) constitue le point d'entrée du programme.

Nous verrons, dans le paragraphe suivant, quelles sont les instructions qui composent le corps d'une fonction. Nous nous intéresserons dans le présent paragraphe aux mécanismes de définition et d'appel des fonctions.

Initialement, dans la définition donnée par Kernighan et Ritchie [1], le mécanisme de déclaration n'offre aucun moyen d'assurer une cohérence entre la déclaration d'une fonction et son appel.

Il était laissé à la charge du programmeur de vérifier que, lors de l'appel d'une fonction, les paramètres étaient bien conformes à ce qui était attendu. En fait, ce laxisme quant à la vérification de la cohérence, a même été utilisé comme une des caractéristiques du langage. Tout programmeur C connaît la fonction `printf`, qui permet d'effectuer une impression formatée et qui admet de par sa description même un nombre d'arguments variables, et dont le type est indiqué par le premier argument et peut donc n'être connu qu'à l'exécution.

Depuis les travaux ANSI, un consensus se dégage pour offrir des mécanismes qui permettent de décharger le programmeur des problèmes de compatibilité entre déclaration et appel de fonctions.

Il est intéressant de noter que, pour rester conforme à l'esprit du langage, le mécanisme choisi est un mécanisme optionnel et qu'il est donc possible en ce qui concerne la norme ANSI d'écrire des programmes dans lesquels les vérifications de paramètres sont inhibées.

Ce mécanisme de vérification des paramètres de fonctions a été salué par beaucoup d'utilisateurs de langages autres que C comme un pas dans la bonne direction afin de rendre l'apprentissage du langage plus facile, et son utilisation plus fiable.

7,1 Définition et portée des fonctions.

La définition d'une fonction C comporte la déclaration de la fonction proprement dite, suivie éventuellement de la déclaration des arguments, suivie par un <bloc-programme> qui décrit l'action de la fonction lorsqu'elle sera appelée. Un <bloc-programme> commence par une accolade ouvrante suivie éventuellement d'une suite de déclarations locales, puis suivie éventuellement par une suite d'instructions et se termine par une accolade fermante.

Comme pour les objets (cf. § 5), la déclaration d'une fonction permet de spécifier son type, c'est-à-dire en fait le type de la valeur qu'elle retourne à son appelant, sa portée ou visibilité, ainsi que ses paramètres.

La portée d'une fonction C peut être limitée au fichier contenant sa définition ; elle est alors dite locale et déclarée avec le mot-clé **static**. Dans le cas contraire, la fonction est dite globale, et sa définition soit ne comporte pas de spécification de visibilité, soit est précédée du mot-clé **extern**. Par défaut, toute fonction C est une fonction globale.

Dans la définition du langage C par Kernighan et Ritchie, le type de valeur retourné par une fonction était restreint aux types simples et aux pointeurs. Dans la norme ANSI, la possibilité pour une fonction de renvoyer une structure a été ajoutée, ainsi que le type **void** qui permet d'indiquer que le résultat retourné par la fonction n'est pas signi-

ficatif, introduisant de ce fait l'équivalent de la notion de procédure telle qu'on la connaît avec la sous-routine en FORTRAN ou la procédure en PASCAL.

Si le type d'une fonction n'est pas clairement spécifié lors de sa déclaration, elle est considérée comme fonction retournant un entier.

C'est lors de la déclaration des paramètres d'une fonction que l'on indique au compilateur si l'on a choisi d'imposer la vérification des paramètres. Ce mécanisme appelé **prototypage** est décrit dans le paragraphe suivant.

7,2 Prototypage.

Le prototypage, rappelons-le, est un acquis des travaux de normalisation du langage C menés par l'ANSI. C'est le mécanisme qui permet de forcer la vérification des paramètres d'appel d'une fonction, ou encore de forcer des conversions implicites lors de l'appel d'une fonction.

Le meilleur moyen d'expliquer ce mécanisme est encore de l'illustrer par un exemple. Une fonction recevant en entrée deux entiers, et retournant leur somme s'écrit (sans prototypage) :

```
int sum(i, j) /* la fonction s'appelle sum et renvoie un entier
              ses paramètres sont i et j */
{
    int i, j; /* les paramètres sont de type entier */
    return (i + j);
}
```

La même fonction, pour laquelle l'on voudra forcer le prototypage, s'écrit :

```
int sum(int i, int j)
{
    return (i + j);
}
```

La différence est dans la façon dont sont déclarés les paramètres. La deuxième forme force le prototypage.

Brièvement résumées, les conséquences du prototypage sont les suivantes : si un argument d'appel ne correspond pas à la déclaration du paramètre correspondant, il y a tentative de conversion ; si la conversion est possible, elle est effectuée de façon implicite, sinon il y a émission d'un message d'erreur par le compilateur.

Il est toujours possible de déclarer des fonctions ayant un nombre de paramètres variable en C ; le paramètre particulier « ... » permet d'indiquer que la fonction a un nombre de paramètres variables, comme dans l'exemple ci-après où la fonction `fonc` a pour paramètres un entier, un flottant puis une suite de paramètres variables :

```
int fonc(int i, float j, ...)
{
    <corps_de_fonction>
}
```

7,3 Appel de fonction.

L'appel d'une fonction se fait en spécifiant son nom suivi d'une parenthèse ouvrante, suivie de la liste optionnelle des arguments d'ap-

pel, le tout terminé par une parenthèse fermante ; la liste des arguments peut s'étendre sur plusieurs lignes.

Les lignes ci-après sont des appels de fonctions valides :

```
func(1, 2);
func(1 + 2 / 3, 2);
```

Selon que la fonction est une fonction prototypée ou non, il y aura vérification et éventuellement conversion des arguments.

L'ordre d'évaluation des paramètres n'est absolument pas défini, et peut être quelconque (néanmoins la pratique montre qu'ils sont le plus souvent évalués de droite à gauche).

Un appel de fonction apparaît généralement dans une expression, sauf en ce qui concerne les fonctions de type **void**.

```
i = sum(1, 3) + sum(2, 1);
```

ou encore :

```
if (sum(i, j) == 3)
    k = 1;
else
    k = 2;
```

Il existe en règle générale deux mécanismes de passation des paramètres :

- par adresse : l'adresse du paramètre est transmise, et c'est donc l'argument lui-même qui est manipulé et éventuellement modifié ; c'est la méthode employée par FORTRAN IV, par exemple ;

- par valeur : l'argument est recopié dans une zone locale, et c'est cette copie qui est transmise ; seule la valeur de l'argument pourra ainsi être atteinte.

En ce qui concerne C, le mécanisme choisi est celui de la passation par valeur, sauf dans le cas des tableaux où il y a passation par adresse (pour des raisons d'efficacité et aussi parce que, en C, la taille d'un tableau n'est pas toujours connue).

Il est néanmoins toujours possible d'effectuer une passation par adresse, il suffit pour cela d'utiliser des variables de type pointeur.

La fonction ci-dessous reçoit en paramètre un entier :

```
int func(int i)
```

```
{
    ...
}
```

et ne pourra en aucun cas en modifier la valeur. Si l'on effectuait une telle opération, la fonction devrait être déclarée de la façon suivante :

```
int func(int *i)
```

```
{
    ...
}
```

la fonction reçoit alors en paramètre un pointeur vers un entier, qu'il est possible de modifier en écrivant par exemple :

```
*i = 0;
```

8. INSTRUCTIONS

Comme on l'a vu au paragraphe 4, la partie algorithmique d'un programme C est constituée d'une suite de fonctions. Toutes les instructions du programme sont regroupées à l'intérieur de fonctions.

La définition d'une fonction comporte un **<bloc-programme>** qui décrit l'action de la fonction lorsqu'elle sera appelée. Un **<bloc-programme>** commence par une accolade ouvrante suivie éventuellement

d'une suite de déclarations locales, puis suivie éventuellement par une suite d'instructions, et se termine par une accolade fermante. En dehors de la description donnée ci-dessus, laquelle peut être utilisée de façon récursive, on dispose du jeu d'instructions suivant.

<expression> : Une expression terminée par un point virgule est une instruction qui provoque l'évaluation de l'expression et l'abandon du résultat. Les affectations et les appels de fonctions sont simplement des cas particuliers de cette définition.

Un point virgule tout seul est une instruction vide. Elle ne fait rien.

if (<expression>) <instruction> [else <instruction>] Si <expression> donne une valeur non nulle, quel que soit son type, l'instruction suivante est exécutée, et celle suivant le **else**, s'il est présent, est sautée. Dans le cas contraire, l'instruction suivante est sautée, et celle suivant le **else**, s'il est présent, est exécutée. Comme dans les autres langages, un **else** dans une suite de **if** emboîtés est associé au **if** libre le plus proche.

switch (<expression>) <instruction> Cette instruction permet d'implanter un choix. Si le résultat de l'expression converti en entier correspond à la valeur associée à une instruction **case** présente dans <instruction>, l'exécution continue juste après le **case** identifié. Sinon, si l'étiquette **default** est présente, l'exécution continue juste après. Sinon, l'instruction du **switch** est sautée. L'instruction d'un **switch** est typiquement un **<bloc-programme>**, mais ce n'est pas obligatoire.

case <valeur> : <instruction> Cette instruction désigne un des choix possibles associés à un **switch**, l'instruction **case** ne peut apparaître que dans la portée d'un **switch**. La valeur doit être une valeur entière calculable à la compilation.

default : <instruction> L'étiquette **default** ne peut apparaître que dans la portée d'un **switch** et au plus une seule fois.

while (<expression>) <instruction> Tant que le résultat de l'expression est une valeur non nulle, quel que soit son type, l'instruction est exécutée. L'expression est évaluée avant chaque exécution de l'instruction, plus une fois si sa valeur est nulle. L'instruction peut ainsi être exécutée zéro ou plusieurs fois.

do <instruction> while (<expression>); L'instruction est exécutée, puis, tant que le résultat de l'expression est une valeur non nulle, quel que soit son type, l'instruction est répétée. L'expression est évaluée après chaque exécution de l'instruction, qui peut ainsi être exécutée une ou plusieurs fois.

for (<ex1> ; <ex2> ; <ex3>) <instruction> <ex1>, <ex2> et <ex3> sont trois expressions ; <ex1> est évaluée exactement une fois puis, tant que le résultat de <ex2> est une valeur non nulle, l'instruction est exécutée et <ex3> est évaluée. L'instruction **for** se comporte exactement comme la séquence suivante :
| <ex1> ; while (<ex2>) | <instruction> <ex3> ; |

break ; Une instruction **break** provoque la sortie immédiate du **switch**, **while**, **do** ou **for** courant et le plus imbriqué. L'exécution reprend à l'instruction suivante. Un **break** ne peut apparaître que dans l'instruction d'un **switch**, **while**, **do** ou **for**.

continue ; Une instruction **continue** provoque la sortie immédiate de la portée d'un **while**, **do** ou **for**. L'exécution reprend au niveau du test pour un **while** ou un **do**, et au niveau de <ex3> pour un **for**. Une instruction **continue** ne peut apparaître que dans l'instruction d'un **while**, **do** ou **for**.

goto <identificateur> ; Une instruction **goto** force la reprise de l'exécution à l'instruction suivant l'étiquette correspondant à l'identificateur présent dans le même bloc. Cette étiquette doit exister.

<identificateur> : <instruction> Une instruction étiquetée sert de point de branchement pour un **goto** décrit précédemment. Les étiquettes d'un même bloc doivent être toutes différentes.

return [<expression>] ; Si l'expression est présente, elle est évaluée puis convertie dans le type retourné par la fonction, puis la fonction retourne au bloc appelant avec cette valeur associée. Si

l'expression est absente, la fonction retourne au bloc appelant avec une valeur indéterminée. Le compilateur ajoute un **return** implicite à la fin du <bloc-programme> de la définition de fonction.

Encore une fois C est un langage simple, donc avec peu d'instructions.

Contrairement à d'autres langages (tel PASCAL), lorsqu'il évalue une expression, par exemple pour un **if** ou un **for**, le langage C arrête l'évaluation dès que le résultat peut être considéré comme acquis. Ainsi, lors de l'évaluation d'une suite de ET logiques, l'expression sera évaluée jusqu'à la rencontre d'un terme faux, les termes suivants ne seront alors même pas évalués. Cette convention permet d'écrire de façon plus condensée, mais prête parfois à confusion pour les utilisateurs habitués aux langages où les expressions sont toujours totalement évaluées.

9. BIBLIOTHÈQUES

Les bibliothèques en C ont tout d'abord servi à pallier les insuffisances du langage tel que défini initialement. Le lecteur averti aura remarqué que jusqu'à présent nous n'avons décrit aucun mécanisme d'entrée-sortie, ou encore d'opérateurs de manipulation de caractères ou de chaînes de caractères.

Toutes ces fonctions en C sont gérées à l'aide de bibliothèques. Une des raisons de non-portabilité des programmes C était l'existence de bibliothèques diverses, et parfois même, propres à différents compilateurs.

Les travaux de normalisation ANSI devraient mettre un terme à cette situation puisque la proposition de norme ANSI inclut une spécification complète des bibliothèques qui doivent accompagner le langage.

Il n'est point dans notre intention de décrire ici les diverses bibliothèques existantes, mais seulement d'en indiquer le champ d'application et la teneur générale.

La bibliothèque communément utilisée par tous les compilateurs est celle décrite dans le guide de référence du programmeur Unix [2]; on en trouvera également une description dans l'ouvrage de T. Hogan [8]; les bibliothèques décrites dans la norme ANSI [3] ont dans la mesure du possible conservé une compatibilité avec la bibliothèque Unix.

Le lecteur intéressé par une description de cette bibliothèque pourra se reporter au manuel correspondant [2].

Étant donné l'influence et le poids qu'aura la norme ANSI, qui risque de devenir rapidement une norme ISO, c'est-à-dire une norme internationale, nous préférons décrire brièvement les bibliothèques de la nor-

me ANSI. L'utilisateur potentiel et désireux de s'affranchir des problèmes de portabilité devrait utiliser les bibliothèques ANSI plutôt que toute autre.

Les fonctions fournies par les diverses bibliothèques sont résumées ci-dessous :

- traitement des caractères,
- fonctions mathématiques en simple et double précision,
- branchement généralisé,
- traitement des signaux, c'est-à-dire des exceptions liées à l'exécution du programme lui-même, ou émises par l'environnement dans lequel s'exécute le programme,
- arguments variables,
- fonctions d'entrée-sortie, telles que accès aux fichiers et fonctions de formatage des données pour visualisation,
- utilitaires généraux, tels que fonctions de conversion d'entiers en divers formats textes ou vice versa.
- traitement des chaînes de caractères telles que comparaison, concaténation, copie,
- fonction d'accès à la date et à l'heure, formatage de la date et de l'heure.

En fait ces diverses bibliothèques contiennent plus d'une centaine de points d'entrée, dont la description n'est pas l'objet du présent article. Le point important à noter est que le contenu des bibliothèques est maintenant un domaine qui possède une description sur laquelle un consensus clair est établi.

10. CONCLUSION

C est un langage qui, petit à petit, s'impose dans de nombreux domaines, et tout naturellement dans son domaine de prédilection celui du logiciel de base. Son succès grandissant, et l'apparition d'une norme qui devrait très rapidement devenir une norme internationale ne fait qu'élargir ses horizons.

S'il est vrai que ce langage a peu à peu quitté le monde des spécialistes pour s'adresser à un nombre de plus en plus grand d'utilisateurs, il n'en est pas moins vrai que la philosophie de base du langage est que le programmeur est roi. A ce titre, il est possible d'écrire des programmes extrêmement performants en C [5], mais il est aussi possible de programmer les pires bêtises, ou encore de produire des programmes difficilement lisibles. Tous ces inconvénients sont facilement contournés à condition de suivre une discipline de programmation adaptée [5].

BIBLIOGRAPHIE

1. KERNIGHAN (B.W.) et RITCHIE (D.). — *The C programming language* (Le langage de programmation C). 228 p. Englewoods Cliffs 1978 Prentice-Hall. (Cet ouvrage est disponible en traduction française sous le titre : Le langage de programmation C).
2. *UNIX programmer reference manual* (Manuel de référence du programmeur) Release 2.0, avr. 1984, A.T. & T. Bell Labs. (Ce document fait partie de la documentation UNIX).
3. *Draft proposed American National Standard Institute for information systems - programming language C* (Proposition de norme pour le langage C). Washington 1983, 1987 American National Standards Institute.
4. PLUM (T.). — *C programming guidelines* (Règles de programmation C). 145 p. Cardiff NJ 1984 Plum Hall.
5. BRODIE (J.) et PLUM (T.). — *Efficient C* (C efficace). 150 p. approx. Cardiff NJ 1985 Plum Hall.
6. PLUM (T.). — *Reliable data structures in C* (Structures de données fiables en C). 250 p. approx. Cardiff NJ 1985 Plum Hall.
7. FEUER (A.) et NARAIN (G.). — *Comparing and assessing programming languages: ADA, C, PASCAL*. Englewoods Cliffs 1984 Prentice Hall.
8. HOGAN (T.). — *The C programmer's handbook*. 262 p. Bowie 1984 Brady Communications Co. Inc. A Prentice-Hall Publ. Co.
9. KELLEY (A.) et POHL (I.). — *A Book on C*. 355 p. Menlo Park 1984 The Benjamin/Cummings Publ. Co. Inc. Reading.
10. WORTMAN (L.A.) et SIDEBOTTOM (T.O.). — *The C programming tutor*. 270 p. London 1984 Prentice-Hall International, Inc., and Robert J. BRADY Co.

On pourra consulter aussi :

7. FEUER (A.) et NARAIN (G.). — *Comparing and*

Toute reproduction sans autorisation du Centre Français du Copyright est strictement interdite.

© Techniques de l'Ingénieur, traité Informatique.

Gagnez du temps et sécurisez vos projets en utilisant une source actualisée et fiable



RÉDIGÉE ET VALIDÉE
PAR DES EXPERTS



MISE À JOUR
PERMANENTE



100 % COMPATIBLE
SUR TOUS SUPPORTS
NUMÉRIQUES



SERVICES INCLUS
DANS CHAQUE OFFRE

- + de 340 000 utilisateurs chaque mois
- + de 10 000 articles de référence et fiches pratiques
- Des Quiz interactifs pour valider la compréhension 

SERVICES ET OUTILS PRATIQUES



Questions aux experts*

Les meilleurs experts techniques et scientifiques vous répondent



Articles Découverte

La possibilité de consulter des articles en dehors de votre offre



Dictionnaire technique multilingue

45 000 termes en français, anglais, espagnol et allemand



Archives

Technologies anciennes et versions antérieures des articles



Info parution

Recevez par email toutes les nouveautés de vos ressources documentaires

*Questions aux experts est un service réservé aux entreprises, non proposé dans les offres écoles, universités ou pour tout autre organisme de formation.

Les offres Techniques de l'Ingénieur



INNOVATION

- Éco-conception et innovation responsable
- Nanosciences et nanotechnologies
- Innovations technologiques
- Management et ingénierie de l'innovation
- Smart city – Ville intelligente



MATÉRIAUX

- Bois et papiers
- Verres et céramiques
- Textiles
- Corrosion – Vieillessement
- Études et propriétés des métaux
- Mise en forme des métaux et fonderie
- Matériaux fonctionnels. Matériaux biosourcés
- Traitements des métaux
- Élaboration et recyclage des métaux
- Plastiques et composites



MÉCANIQUE

- Frottement, usure et lubrification
- Fonctions et composants mécaniques
- Travail des matériaux – Assemblage
- Machines hydrauliques, aérodynamiques et thermiques
- Fabrication additive – Impression 3D



ENVIRONNEMENT – SÉCURITÉ

- Sécurité et gestion des risques
- Environnement
- Génie écologique
- Technologies de l'eau
- Bruit et vibrations
- Métier : Responsable risque chimique
- Métier : Responsable environnement



ÉNERGIES

- Hydrogène
- Ressources énergétiques et stockage
- Froid industriel
- Physique énergétique
- Thermique industrielle
- Génie nucléaire
- Conversion de l'énergie électrique
- Réseaux électriques et applications



GÉNIE INDUSTRIEL

- Industrie du futur
- Management industriel
- Conception et production
- Logistique
- Métier : Responsable qualité
- Emballages
- Maintenance
- Traçabilité
- Métier : Responsable bureau d'étude / conception



ÉLECTRONIQUE – PHOTONIQUE

- Électronique
- Technologies radars et applications
- Optique – Photonique



TECHNOLOGIES DE L'INFORMATION

- Sécurité des systèmes d'information
- Réseaux Télécommunications
- Le traitement du signal et ses applications
- Technologies logicielles – Architectures des systèmes
- Sécurité des systèmes d'information



AUTOMATIQUE – ROBOTIQUE

- Automatique et ingénierie système
- Robotique



INGÉNIERIE DES TRANSPORTS

- Véhicule et mobilité du futur
- Systèmes aéronautiques et spatiaux
- Systèmes ferroviaires
- Transport fluvial et maritime



MESURES – ANALYSES

- Instrumentation et méthodes de mesure
- Mesures et tests électroniques
- Mesures mécaniques et dimensionnelles
- Qualité et sécurité au laboratoire
- Mesures physiques
- Techniques d'analyse
- Contrôle non destructif



PROCÉDÉS CHIMIE – BIO – AGRO

- Formulation
- Bioprocédés et bioproductions
- Chimie verte
- Opérations unitaires. Génie de la réaction chimique
- Agroalimentaire



SCIENCES FONDAMENTALES

- Mathématiques
- Physique Chimie
- Constantes physico-chimiques
- Caractérisation et propriétés de la matière



BIOMÉDICAL – PHARMA

- Technologies biomédicales
- Médicaments et produits pharmaceutiques



CONSTRUCTION ET TRAVAUX PUBLICS

- Droit et organisation générale de la construction
- La construction responsable
- Les superstructures du bâtiment
- Le second œuvre et l'équipement du bâtiment
- Vieillessement, pathologies et réhabilitation du bâtiment
- Travaux publics et infrastructures
- Mécanique des sols et géotechnique
- Préparer la construction
- L'enveloppe du bâtiment
- Le second œuvre et les lots techniques